NeROIC: Neural Rendering of Objects from Online Image Collections

Zhengfei Kuang *
University of Southern California

Kyle Olszewski
Snap Inc.

Menglei Chai
Snap Inc.

Zeng Huang
Snap Inc.

Panos Achlioptas
Snap Inc.

Sergey Tulyakov
Snap Inc.

Abstract

We present a novel method to acquire object representations from online image collections, capturing high-quality geometry and material properties of arbitrary objects from photographs with varying cameras, illumination, and backgrounds. This enables various object-centric rendering applications such as novel-view synthesis, relighting, and harmonized background composition from challenging in-the-wild input. Using a multi-stage approach extending neural radiance fields, we first infer the surface geometry and refine the coarsely estimated initial camera parameters, while leveraging coarse foreground object masks to improve the training efficiency and geometry quality. We also introduce a robust normal estimation technique which eliminates the effect of geometric noise while retaining crucial details. Lastly, we extract surface material properties and ambient illumination, represented in spherical harmonics with extensions that handle transient elements, e.g. *sharp shadows. The union of these components results in a highly modular and efficient object acquisition framework. Extensive evaluations and comparisons demonstrate the advantages of our approach in capturing high-quality geometry and appearance properties useful for rendering applications. Our code will be released at* https://formyfamily.github.io/NeROIC/

1. Introduction

Numerous collections of images featuring identical objects, *e.g.* furniture, toys, vehicles, can be found online on shopping websites or through a simple image search. The ability to isolate these objects from their surroundings and capture high-fidelity structure and appearance is highly desired, as it would enable applications such as digitizing an object from the images and blending it into a new background. However, individual images of the objects in these collections are typically captured in highly variable backgrounds, illumination conditions, and camera parameters, making object digitization approaches specifically designed for data from controlled environments unsuitable for such an *in-the-wild* setup. In this work, we seek to address this challenge by developing an approach for capturing and re-rendering objects from unconstrained image collections by extending the latest advances in neural object rendering.

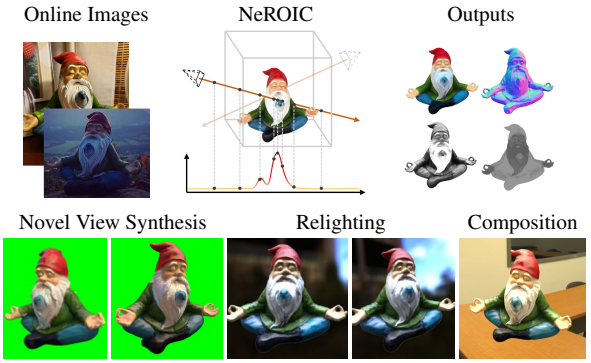


Figure 1. **Our Object Capture Results from Online Images.** Our modular NeRF-based approach requires only sparse, coarsely segmented images depicting an object captured under widely varying conditions (top left). We first infer the geometry as a density field using neural rendering (top right), and then compute the object's surface material properties and per-image lighting conditions (middle). Our model not only can synthesize novel views, but can also relight and composite the captured object in novel environments and lighting conditions (bottom).

Among the more notable recent works using implicit 3D scene representations is the Neural Radiance Fields (NeRF) model [22], which learns to represent the local opacity and view-dependent radiance of a static scene from sparse calibrated images, allowing high-quality novel view synthesis (NVS). While substantial progress has been made to improve the quality and capabilities of NeRF (*e.g.* moving or non-rigid content [14, 26, 29, 41], some non-trivial requirements still remain – to synthesize novel views of an object the background and illumination conditions should be seen and fixed, and the multi-view images or video sequences

*This work was performed while the author was an intern at Snap, Inc.

should be captured in a single session.

Recently, several works [3, 4, 6, 20, 43, 49] have extended NeRF and achieved impressive progress in decomposing the renderings of a scene into semantically meaningful components, including geometry, reflectance, material, and lighting, enabling a flexible interaction with any of these components, *e.g.* relighting and swapping the background. Unfortunately, none of them built a comprehensive solution to work with the limitations of objects captured from real-world, in-the-wild image collections. In this work, we propose **NeROIC**, a novel approach to **Ne**ural **R**endering of objects from **O**nline **I**mage **C**ollections. Our object capture and rendering approach builds upon neural radiance fields with several key features that enable high-fidelity capture from sparse images captured under wildly different conditions, which is commonly seen in online image collections with individual images taken with varying lightings, cameras, environments, and poses. The only expected annotation for each image is a rough foreground segmentation and coarsely estimated camera parameters, which crucially we can obtain in an unsupervised, and cost-free way from structure-from-motion frameworks such as COLMAP [33].

Key to our learning-based method is the introduction of a modular approach, in which we first optimize a NeRF model to estimate the geometry and refine the camera parameters, and then infer the surface material properties and per-image lighting conditions that best explain the captured images. The decoupling of these stages allows us to use the depth information from the first stage to do more efficient ray sampling in the second stage, which improves material and lighting estimation quality and training efficiency. Furthermore, due to the modularity of our approach we can also separately exploit the surface normals initialized from the geometry in the first stage, and innovate with a new normal extraction layer that enhance the accuracy of acquiring materials of the underlying object. An overview of our approach is shown in Fig. 2 (b).

To evaluate our approach, we create several in-the-wild object datasets, including images captured by ourselves in varying environments, as well as images of objects collected from online resources. The comparisons with state-of-the-art alternatives, in these challenging setups, indicate that our approach outperforms the alternatives qualitatively and quantitatively, while still maintaining comparable training and inference efficiency. Fig 1 presents a set of example object capturing and application results by our approach.

In summary, our main contributions are:

- A novel, modular pipeline for inferring geometric and material properties from objects captured under varying conditions, using only sparse images, foreground masks, and coarse camera poses as additional input,
- A new multi-stage architecture where we first extract the geometry and refine the input camera parameters, and then infer the object's material properties, which we show is robust to unrestricted inputs,
- A new method for estimating normals from neural radiance fields that enables us to better estimate material properties and relight objects than more standard alternative techniques,
- Datasets containing images of objects captured in varying and challenging environments and conditions,
- Extensive evaluations, comparisons and results using these and other established datasets demonstrating the state-of-the-art results obtained by our approach.

We will release our code, pre-trained models, and training datasets upon publication to facilitate further research effort in this area.

2. Related Work

Neural Rendering for Novel View Synthesis. One of the more recent advances in novel view synthesis is NeRF [22]. A set of multilayer perceptrons (MLPs) are used to infer the opacity and radiance for each point and outgoing direction in the scene by sampling camera rays and learning to generate the corresponding pixel color using volume rendering techniques, allowing for high-quality interpolation between sparse training images. However, this framework requires well-calibrated multi-view datasets of static scenes as input, with no variation in the scene content and lighting conditions. Many subsequent works build upon this framework to address these and other issues. NeRF– [40], SCNeRF [8] and BARF [16] infer the camera parameters while learning a neural radiance field, to allow for novel view synthesis when these parameters are unknown. iNeRF [44] recovers ground-truth poses by inverting a trained neural radiance field to render the input images. Other works focus on improving the training or inference performance and computational efficiency [17, 18, 23, 31, 39, 45]). Related approaches [1, 10] use a signed-distance function to represent a surface that can be extracted as a mesh for fast rendering and novel view synthesis. However, these works only display high-quality results for a limited range of interpolated views, and do not perform the level of material decomposition and surface reconstruction needed for high-quality relighting and reconstruction.

Learning from Online Image Collections. Online image collections have been used for various applications, such as reconstructing the shape and appearance of architectures [35, 36] or human faces [11, 15]. However, such approaches typically require many available photographs, making them applicable only to landmarks or celebrities,

and are designed explicitly to work with specific subjects with domain features, rather than arbitrary objects. Recently, neural rendering has been combined with the use of generative adversarial networks [5] to allow for generatively sampling different objects within a category and rendering novel views [24, 25, 34], using only a single image of each training object. However, such approaches do not allow for rendering novel views of a target object, *e.g.* from one or more images, with controllable shape, appearance, and environmental conditions, and the quality of the sampled images varies. Other approaches learn pose, shape, and texture from images for certain categories of objects [7, 21], or interpolation, view synthesis, and segmentation of sampled category instances [42]. However, none of these approaches allow for the level of structure and material decomposition suitable for high-fidelity rendering and relighting.

Image Content Decomposition and Relighting. Many recent works focus on decomposing the lighting condition and intrinsic properties of the objects from the training images. NeRF-in-the-Wild [20] learns to render large-scale scenes from images captured at different times by omitting inconsistent and temporary content, such as passersby, and implicitly representing lighting conditions as appearance features that can be interpolated. However, this approach does not fully decompose the scene into geometric and material properties for arbitrary lighting variations, and is not designed to address challenging cases such as extracting isolated objects from their surroundings. On the other hand, many works including Neural Reflectance Field [2], NeRFactor [49], NeRV [37], and PhySG [47] combine NeRF with physical-based rendering techniques, and estimate various material properties of the target object. However, all of these works require well-conditioned or known lighting, and are not adaptive to input images from unknown arbitrary environments. Some recent works, i.e. NeRD [3], Neural-PIL [4] and NeRS [46] relax the constraint of dataset, but they still requires restrictions on the inputs, such as known exposure and white balancing parameters [3], data from the same source [46]. Most importantly, all of these approaches are inevitably vulnerable to inputs with complex shading, i.e. sharp shadows and mirror-like reflections, since they only consist of one physical-based renderer which is relatively simple. While we do not take the claim to learn how to fit those shadings in our method, in our work we introduce a transient component based on [49] to identify and disentangle it from environment lighting, thus acquiring unbiased material properties of the object. To the best of our knowledge, we are the first NeRF-based method to infer both geometry and material parameters of the target with *fully unconstrained images from the internet.*

3. Method

In this section, we outline our approach to object-centric aggregation. We first provide an overview of the approach (Sec. 3.1), followed by a description of the neural radiance fields framework we extend in our method (Sec. 3.2).

3.1. Overview

Fig. 2 provides an overview of our approach. The inputs are a sparse collection of images $\mathcal{I}_k : [0,1]^2 \rightarrow [0,1]^3$ depicting an object (or instances of an identical object) under varying conditions, and a set of foreground masks $\mathcal{F}_k : [0,1]^2 \rightarrow \{0,1\}$ defining the region of the object, where $1 \leq k \leq N$. During the first stage, we estimate the geometry of the object by learning a density field indicating where there is physical content (Sec. 3.3). During this stage, we also learn both static and transient radiance values to allow for image-based supervision, but do not fully decompose this information into material and lighting properties. We also optimize the pose and intrinsic parameters of the cameras to refine the coarse estimates provided as input.

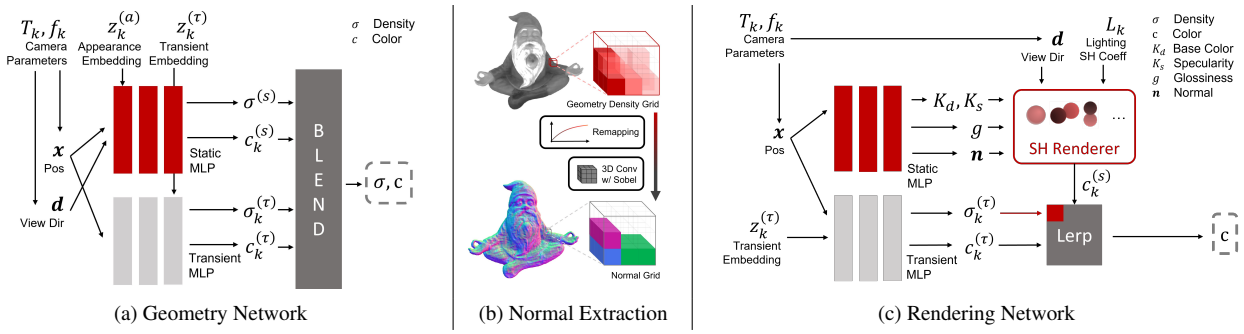
In the second stage, we fix the learned geometry and optimize the surface material and lighting parameters needed to re-render the object in arbitrary illumination conditions (Sec. 3.5). During this stage, we use the estimated distance from the camera to the object surface to improve our point sampling along the camera rays. We also optimize the surface normals, which improves on the coarse estimates that are obtained from our density field (Sec. 3.4).

3.2. Prelimilaries

In Neural Radiance Fields (NeRF) [22], a set of networks are trained to infer radiance and density for arbitrary 3D points, and generate images from novel viewpoints using volumetric rendering. Specifically, it employs two MLP functions: a density function $\sigma(\boldsymbol{x}) : \mathbb{R}^3 \rightarrow \mathbb{R}^+$ and a color function $c(\boldsymbol{x}) : \mathbb{R}^3 \rightarrow [0,1]^3$. For each ray $\boldsymbol{r} = (\boldsymbol{r}_o, \boldsymbol{r}_d)$ emitted from the camera origin $\boldsymbol{r}_o$ in direction $\boldsymbol{r}_d$, NeRF samples N_p 3D points along the ray $\boldsymbol{x}_i = \boldsymbol{r}_o + d_i \boldsymbol{r}_d$ $(0 \leq i \leq N_p)$, and integrates the pixel color as follows:

$$C(\boldsymbol{r}) = \sum_{i=1}^{N_p} \alpha_i (1 - w_i) c(\boldsymbol{x}_i), \tag{1}$$

where $w_i = \exp(-(d_i - d_{i-1})\sigma(\boldsymbol{x}_i))$ represents the transmittance of the ray segment between sample points $\boldsymbol{x}_{i-1}$ and $\boldsymbol{x}_i$, and $\alpha_i = \prod_{j=1}^{i-1} w_i$ is the ray attenuation from the origin $\boldsymbol{r}_o$ to the sample point $\boldsymbol{x}_i$. In addition to the volumetric rendering function, NeRF also introduces an adaptive coarse-to-fine pipeline which uses a coarse model to guide the 3D point sampling of the fine model.



(a) Geometry Network (b) Normal Extraction (c) Rendering Network

Figure 2. **Overview of Our Approach.** Given a set of coarsely calibrated images and corresponding foreground masks, our geometry network computes a neural radiance field with both static and transient components, and refines the camera parameters (a). Our grid-based normal extraction layer then estimates the surface normals from the learned density field (b). Finally, we fix the geometry of the object and use the estimated normals as supervision in our rendering network, in which we infer the lighting conditions (represented as spherical harmonics coefficients), surface material properties (using the Phong rendering model), and high-quality surface normals (c).

3.3. Geometry Networks

In our first stage, we seek to reconstruct the geometry of the target object depicted in our image collection. This, however, is made more challenging due to the varying lighting environments, transient conditions *e.g.* sharp shadows, varying camera parameters, and coarse camera poses and intrinsics caused by the lack of background context required for accurate camera calibration. Inspired by [20], we employ a pipeline designed to make use of images captured under different conditions, and introduce additional designs to account for the challenging task of aggregating an object representation solely from the isolated foreground region.

Thus, we employ a two-branch pipeline which handles transient and static content separately, and assigns unique embedding vectors $\boldsymbol{z}_k^{(\tau)}$ and $\boldsymbol{z}_k^{(a)}$ to each image to represent the transient geometry and changing lighting. Our model for this stage thus consists of four functions instead of two: $\sigma_k^{(s)}(\boldsymbol{x}), \sigma_k^{(\tau)}(\boldsymbol{x})$, and $c_k^{(s)}(\boldsymbol{x}), c_k^{(\tau)}(\boldsymbol{x})$. The volumetric rendering function in [22] is re-formulated as:

$$C_k(\boldsymbol{r}) = \sum_{i=1}^{N_p} \alpha_{ki}((1-w_{ki}^{(s)})c_k^{(s)}(\boldsymbol{x}_i) + (1-w_{ki}^{(\tau)})c_k^{(\tau)}(\boldsymbol{x}_i)), \quad (2)$$

where $w_{ki}^{(s,\tau)} = \exp(-(d_i - d_{i-1})\sigma_k^{(s,\tau)}(\boldsymbol{x}_i))$, and $\alpha_{ki} = \prod_{j=1}^{i-1} w_{ki}^{(s)} w_{ki}^{(\tau)}$. We also adopt the Bayesian learning framework of [12], predicting an uncertainty $\beta_k(x)$ for transient geometry when accounting for the image reconstruction loss.

Eq. 2 serves as the rendering function when training this network. As in [20], we use a color reconstruction loss $\mathcal{L}_c$ incorporated with β_k, and a transient regularity loss $\mathcal{L}_{tr}$. [1]

[1]For details on these losses and their use, please see the supplementary document.

However, to accurately capture the geometry corresponding to our target object, we found it essential to incorporate additional losses designed for our particular use case.

Silhouette Loss. We use the input foreground masks to help the networks focus on the object inside the silhouette, thus preventing ambiguous geometry from images with varying backgrounds. While we mask out the background in each image and replace it with pure white, a naive approach will still fail to discriminate the object from the background, thus producing white artifacts around the object and occluding it in novel views. To avoid this issue, we introduce a silhouette loss $\mathcal{L}_{sil}$, defined by the binary cross entropy (BCE) between the predicted ray attenuation α_k and the ground-truth foreground mask $\mathcal{F}_k$ to guide the geometry learning process. As seen in the ablation study in Tab. 2, the silhouette loss significantly improves our results on testing data.

Adaptive Sampling. We also introduce an adaptive sampling strategy in our model using these masks. At the beginning of every training epoch, we randomly drop out part of the background rays from the training set, to ensure that the ratio of the foreground rays is above $1/3$. This seemingly simple strategy significantly increases the training efficiency, and balances the silhouette loss and prevents α_k from converging to a constant. Our ablation study in Tab. 2 demonstrates that without this adaptive sampling, the model produces much worse results during testing.

Camera Optimization. While our input images come from multiple sources, the lack of a consistent background leads to poor camera pose registration. In practice, though we use COLMAP [33] on images with the backgrounds removed, the poses for some objects are still inaccurate, as seen in Fig. 3. To address this issue, we jointly optimize the camera poses during training, in a manner similar to [40]. More specifically, we incorporate camera parameters $(\delta_R, \delta_t, \delta_f)$

Figure 3. **Comparison on Camera Optimization.** The model trained without camera optimization produces object geometry and color of poorer quality than the full model.

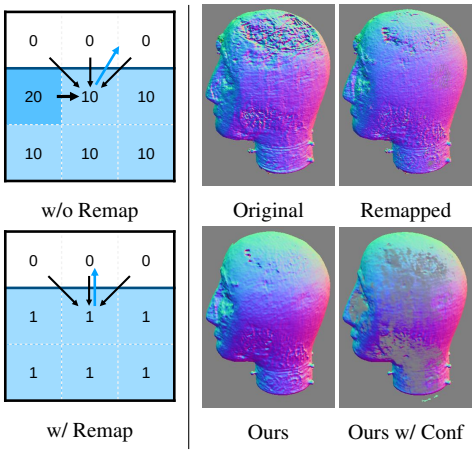


Figure 4. **Analysis of Normal Extraction Layer.** On the left, while the gradient-based normal prediction (blue arrow) may be affected by noise in an unbounded density field, this effect can be alleviated by density remapping ($\lambda = 1$ in this case). On the right, we show the estimated normals from the original density field (top left), remapped normals (top right), our normal extraction layer output (bottom left), and our result with confidence (bottom right).

for rotation, translation, and focal length, respectively. We use an axis-angle representation for rotation, while the others are in linear space. We also add a regularity loss $\mathcal{L}_{cam}$ for the camera parameters, which is simply an L2 loss on these parameters.

As a summary, the final loss we use for this stage is:

$$\mathcal{L}_{geo} = \mathcal{L}_c + \lambda_{tr}\mathcal{L}_{tr} + \lambda_{sil}\mathcal{L}_{sil} + \lambda_{cam}\mathcal{L}_{cam}, \quad (3)$$

where the weights λ_{tr}, λ_{sil}, and λ_{cam} are 0.01, 0.1, and 0.01, respectively, in our experiments.

3.4. Normal Extraction Layer

With the learned geometry from our first stage, we then extract the surface normals of the object as the supervision to the next stage, which helps reduce the ambiguity of the lighting and material estimation task. While many previous works [2, 3, 49] choose to use the gradient of the density function (i.e., $\nabla\sigma^{(s)}(\boldsymbol{x})$) as an approximation of normals, we find that this approach may produce incorrect results in certain areas, due to the challenging issues with unconstrained, real data (blurry images, varying lighting) that reduce the geometry quality and introduce noise into the density function. As explained in Fig. 4, this noise can drastically mislead the normal estimation without changing the surface shape itself. To resolve this, we propose a novel normal estimating pipeline based on the remapping of the density function and 3D convolution on a dense grid, which can produce smooth and accurate normals even with defective density.

We first calculate the bounding box of the object. To do so, we sparsely sample pixels of training images that are inside the foreground mask, and extract the expected surface-ray intersections for each ray, gathered as a point cloud. We directly compute the bounding box on it. After that, we discretize the bounding box into a 512^3 dense grid and extract the density of each grid center. For a grid center $\boldsymbol{x}$, we remap its density value as:

$$\sigma'_{\boldsymbol{x}} = \frac{1}{\lambda}(1 - \exp(-\lambda\sigma_{\boldsymbol{x}})). \quad (4)$$

This function remaps the density value from $[0, +\infty]$ to $[0, \frac{1}{\lambda}]$. The derivative gradually decays as the density value increases, which helps to filter out noise and obtain smoother predictions. λ is a controllable parameter to adjust the sharpness of the normal. As λ decreases, this remapping function converges to the identity function. After remapping, we estimate the gradient of the density field $d\sigma'/d\mathbf{x}$ by applying a 3D convolution with a Sobel kernel $\mathcal{K}(\mathbf{x}) = \mathbf{x}/||\mathbf{x}||_2^2$ of size 5 to the density grid.

Finally, we divide the convolution output $\boldsymbol{n}_{\boldsymbol{x}}^{(g)} = -\mathcal{K}(\sigma'_{\boldsymbol{x}})$ by $\max(1, ||\boldsymbol{n}_{\boldsymbol{x}}^{(g)}||_2^2)$, producing a normal supervision vector with length no larger than 1. We treat its length as the confidence of the estimation, which becomes the weight of its supervising loss in the following stage. We show the results of each step in Fig. 4.

3.5. Rendering Networks

The purpose of our final stage is to estimate the lighting of each input image and the material properties of the object, given the geometry shape and surface normals from previous stages. Since extracting object materials in unknown lighting is highly ill-posed [30,49], we use low-order Spherical Harmonics (SH) to represent our lighting model and optimize its coefficients. However, we use the standard Phong BRDF [28] to model the object material properties, which are controlled by three parameters: K_d for the base color, K_s for the specularity and g for the glossiness. According to [30], this light transportation between a Phong BRDF surface and a SH environment map can be efficiently approximated, and we thus employ these rendering equations in our pipeline.

Hybrid Color Prediction using Transience. Although the spherical harmonics illumination model typically works well on scenes with ambient environment illumination, it lacks the ability to represent sharp shadows and shiny highlights from high-frequency light sources. While we believe it is quite impractical to acquire high-frequency details of lighting and material with respect to our unconstrained input, we hope to eliminate the effect caused by those components, and to learn an unbiased result at lower frequencies. To achieve that, we introduce a hybrid method that combines color prediction with neural networks and parametric models. As in the geometry network described in Sec. 3.3, we employ the concept of *transience*. However, here we do not learn a separate transient geometry in this model, as our geometry is fixed at this point. We use the volumetric rendering in Eq. 1, but replace the color function with:

$$c_k(\boldsymbol{x}) = \text{lerp}\big(c_k^{(\tau)}(\boldsymbol{x}), c^{(SH)}(\boldsymbol{x}), \exp(-\sigma_k^{(\tau)}(\boldsymbol{x}))\big). \quad (5)$$

where $c^{(SH)}(\boldsymbol{x})$ is the output color of our SH renderer.

Estimated Depth for Acceleration. Compared to our geometry networks where color is predicted by neural networks, the rendering stage requires more computation to calculate the color of each sample point due to the more complex rendering equations. On the other hand, however, the learned geometry from the first networks can be used to filter out sampling points that are far away from the object, thus accelerating the whole training process. We develop a hybrid sampling strategy that can speed up the training without introducing any significant artifact.

For a group of N_p sample points $\boldsymbol{x}_i = \boldsymbol{r}_o + d_i\boldsymbol{r}_d$ on a ray, we build a discrete distribution along the ray with the probability of each point proportional to $\alpha_i(1 - w_i)$. Then, we calculate the expectation and variance on d_i w.r.t. to this distribution, denoted as $E(d)$ and $V(d)$. If the variance $V(d)$ is smaller than a threshold τ_d, we then calculate the 3D points at depth $E(d)$ and only use this point for the color calculation. Otherwise, we use all sample points. Please refer to our supplementary material for more details.

Neural Normal Estimation w/ Supervision. Our networks also predict the final surface normals $\boldsymbol{n}(\boldsymbol{x})$, supervised by the output of the normal extraction layer in Sec 3.4, with the reconstruction loss $\mathcal{L}_n$ defined by:

$$\mathcal{L}_n = \Big\| (||\boldsymbol{n}_x^{(g)}||_2) \cdot \boldsymbol{n}(x) - \boldsymbol{n}_x^{(g)} \Big\|_2^2, \quad (6)$$

We also adopt the normal smoothing loss $\mathcal{L}_{sm}$ in [49] to improve the smoothness of the predicted normals.

Additionally, to reduce the ambiguity between the material properties and the lighting, we also add a regularity loss L_{reg} on both the SH coefficients and material properties. Please refer to our supplementary for more details.

In summary, the total loss of this stage is defined as:

$$\mathcal{L}_{render} = \mathcal{L}_c + \lambda_{tr}\mathcal{L}_{tr} + \lambda_n\mathcal{L}_n + \lambda_{sm}\mathcal{L}_{sm} + \mathcal{L}_{reg}, \quad (7)$$

where the weightsλ_{tr}, λ_n, and λ_{sm} are set to 1, 5, and 0.5, respectively, in our experiments.

4. Evaluations

4.1. Implementation details

Training. We use a modified version of MLP structure following [20, 22] as our networks. In the training, We use the Adam optimizer [13] to learn all of our parameters, and our initial learning rate is set to $4 \cdot 10^{-4}$. Our training and inference experiments are implemented using the PyTorch framework [27]. We train our model on 4 NVIDIA V100s with the batch size of 4096, and test our model on a single NVIDIA V100. In the first stage, we train our model with 30 epochs (60K-220K iterations), in roughly 6 to 13 hours. For the second stage, approximately 2 to 4 hours are required for 10 epochs.

Datasets. As our target data is online image collections, all of our evaluations in the paper use real-world datasets. Our datasets are from three different sources: Image Collection from the internet; Objects we captured ourselves; and Realistic data published in NeRD [3]. For the first part, we downloaded user review images of two popular products (named as *Dog* and *Gnome2* in the following context) on Amazon and Taobao, two of the most widely-used online shopping websites; For our self-captured data, we captured 3 objects (*Milk*, *Figure*, *TV*) using our own devices, where each object is placed in about 4-6 different scenes. Finally, we obtained the dataset from NeRD's project website and used 4 of its real object image collections (*Gnome*, *Head*, *Cape*, *MotherChild*) in our evaluation.

For all of the self-collecting datasets, approximately 40 images are collected for each object. Then, we use the SfM pipeline in COLMAP [33] to register the initial camera poses, with image matches generated from SuperGlue [32]. The foreground masks are calculated using the online mask extraction pipeline of remove.bg [9].

4.2. Comparisons

We first show comparisons between our model and NeRF [22] in 7 offline captured objects (Milk, Figure, TV, Gnome, Head, Cape, MotherChild). We adopt the commonly used metrics Peak Signal-to-Noise Ratio (PSNR) and Structural Similarity Index Measure (SSIM) in our evaluation. Considering that the illuminating condition of testing images is unknown, we purpose two settings to evaluate our model: We pick the lighting parameters (either the embedded vector or the SH coefficient) from another training

Table 1. **Comparison with NeRF [22].** We report the average PSNR and SSIM scores of all tested scenes. We also ablate on w/ only the first stage of our approach (Ours-Geom) and w/o optimization (w/o Opt). We highlight the best and second best results of each column in orange and yellow, respectively.

Methods	Mean PSNR ↑	Mean SSIM↑
NeRF [22]	22.266	0.884
Ours-Geom w/o Opt	23.454	0.909
Ours-Full w/o Opt	24.065	0.901
Ours-Geom	26.510	0.919
Ours-Full	26.437	0.909

image in the same scene, and we freeze the networks and optimize the lighting parameters with a Stochastic Gradient Descent optimizer for 1000 steps. As shown in Tab. 1 and Fig. 5, our method in both settings outperforms NeRF by a considerable margin. In qualitative results, our model generates more consistent and smooth results than NeRF does. Aside from the comparison with NeRF, we would like to highlight that our rendering networks produce competitive results compared to the first stage, while it also supports relighting on unseen environments.

In addition to experiments with NeRF, we conducted extensive comparisons with the published results and training data of NeRD [3]. However, as its trained models or training and inference code are not publicly available at the time of our submission, it was intractable to reproduce their method and have a fair quantitative or qualitative comparison with them under the same settings. After careful consideration, we decided to move these experiments and results to our supplementary document, where we demonstrate our model outperforming NeRD in a less direct manner.

4.3. Ablations

To help understand the inevitability and effectiveness of our contributions, we further conduct two ablative studies on our model.

In the first study, we compare our model with three variants on the MotherChild dataset: Model trained without silhouette loss (*Model w/o sil*); Model trained without adaptive sampling (*Model w/o ada*); and Model trained without transient component (*Model w/o tr*). We report the PSNR score and the mean squared error between the attenuation map and the foreground mask (denoted as MMSE) for all models. Results are shown in Tab. 2. While the results of the first two variants significantly degraded in all evaluations, removing the transient component from the model did not significantly affect its performance on the PSNR metric, and we believe this is because in most images the area of occlusion and incorrect mask is relatively small compared to the object, thus having minor effect to the color reconstruction. However, it still decreases the accuracy of geometry

Figure 5. **Qualitative Comparisons with NeRF.** Result of *Ours-Full* are rendered with SH rendering only. Some shadows and highlights are handled as transient component, thus not appearing in *Ours-Full*.

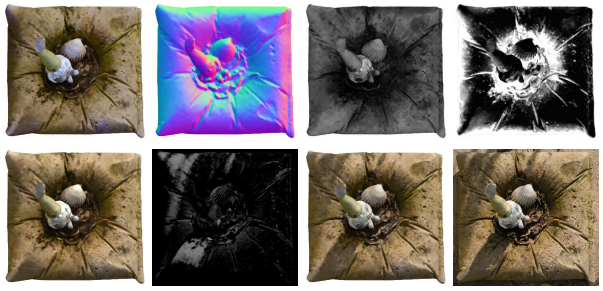


Figure 6. **Examples of Our Decomposition Results.** *First row*: from left to right: diffuse albedo map, normal map, specularity map, and Glossiness map. *Second row*: from left to right: color rendering w/o transient, transient blending weight, color rendering w/ transient, and ground-truth image.

silhouette by one order of magnitude.

Our second ablation study aims to prove that our specific approach generates smooth and accurate material properties in the second stage. We qualitatively compare our full model with three variants on TV and Head datasets: model trained with the original normal from the density field as supervision (*Model w/ on*); model trained without transient component (*Model w/o tr*) and model trained without regularity loss (*Model w/o reg*). As shown in Fig. 7, using normal without proper processing will result in worse normal prediction in both data; removing transient component will consequent to the unwanted artifacts on TV's smooth surface, where mirroring reflections are more likely to appear; and model without regularity loss outputs a biased albedo

in Head data. Our full model tackles all of these problems and produces the most appropriate results.

Table 2. **Ablation Study on Geometry Networks**. From top to bottom: model trained without silhouette loss; model trained without adaptive sampling; model trained without transient model; full model. Please notice that MMSE is not affected by model optimization since it is a geometry-based metric.

Methods	PSNR w/o opt↑	PSNR↑	MMSE↓
Model w/o sil	21.30	23.38	0.18
Model w/o ada	13.60	13.78	0.073
Model w/o tr	21.34	30.27	0.03
Full model	23.03	28.98	0.003

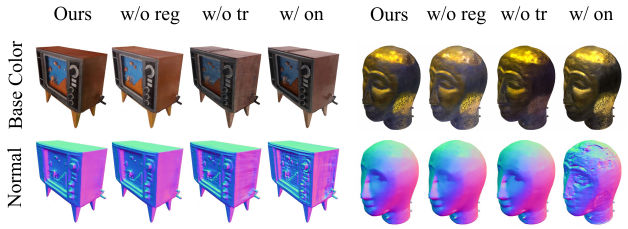


Figure 7. **Ablation Study on Rendering Networks.** We show diffuse albedo maps and normal maps predicted by our models. We increase the exposure of albedo maps for the Head data since the black area in the original outputs is extremely dim.

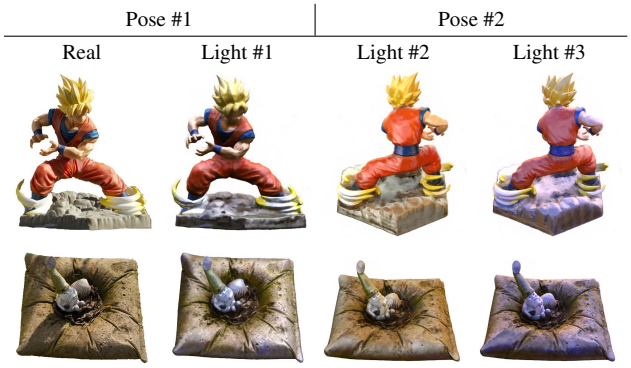


Figure 8. **Sample Relighting Results.** Left: comparison between our relit result and a real image; Right: Model relit in another pose.

4.4. Relighting and Compositing Results

We provide more results of our model in three different showcases: material decomposition, relighting, and composition with online objects.

Fig. 6 shows our prediction of material properties and rendering components for the Gnome dataset. We would like to highlight that our model successfully disentangled the shadow in the input images from our SH rendering component, and learned unbiased material properties from the whole training set. With the material properties, we are able to re-render the objects with new lighting environments, and the results are shown in Fig. 8.

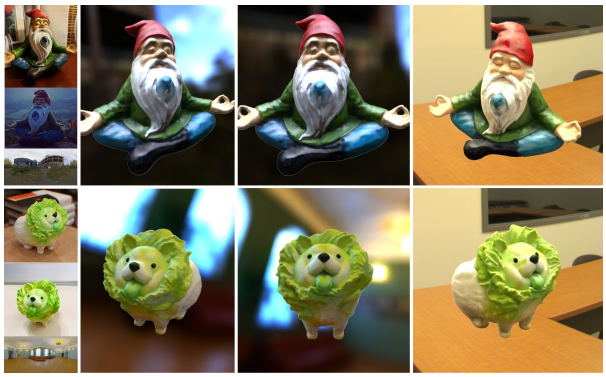


Figure 9. **Compositing Results.** Examples of the input online images and the environment maps are shown on the left. Our composition results are shown on the right.

Finally, we demonstrate results for our target application, rendering objects from online image collections in novel environments and lighting. With images of several items collected from the internet, we can recover their geometry and material properties, and finally re-render and compose them into a new environment. As shown in Fig. 9, even though our input images are captured in vastly differing environments, our model handles this challenging task, producing high-quality and plausible compositing results.

5. Conclusion

In this paper, we demonstrate that compelling capture, compositing, and relighting results are possible using only online image collections for which calibrated multi-view datasets captured in controlled settings are unavailable. This opens the door for many interesting future applications, such as re-rendering objects that may one day be rare or non-extant, represented in online image collections but never explicitly captured using multi-view techniques.

Limitations. Our approach has a few limitations. Although it can process complex and challenging shading in input images, *e.g.* sharp shadows, it does not support generating such components in novel scenes. While some works [37,49] uses terms such as light visibility to represent these effects in a physically-based way, applying these techniques to data with varying unknown illumination is much more challenging, and requires further investigation. We are planning to explore this topic in our future work.

Ethical Implications. As concerns about the potential harm of AI technology are rising rapidly, we feel obliged to point out possible negative impacts of our work. As we aim to acquire accurate geometry and material from online images, it may be improperly used to reconstruct objects that are private or sensitive to the public, *e.g.* unclothed human bodies, or private objects or locations. We believe such risks can be addressed with sufficient legal supervision.

References

[1] Alexander W. Bergman, Petr Kellnhofer, and Gordon Wetzstein. Fast training of neural lumigraph representations using meta learning. In *NeurIPS*, 2021. 2

[2] Sai Bi, Zexiang Xu, Pratul Srinivasan, Ben Mildenhall, Kalyan Sulkavalli, Miloš Hašan, Yannick Hold-Geoffroy, David Kriegman, and Ravi Ramamoorthi. Neural reflectance fields for appearance acquisition. *https://arxiv.org/abs/2008.03824*, 2020. 3, 5

[3] Mark Boss, Raphael Braun, Varun Jampani, Jonathan T. Barron, Ce Liu, and Hendrik Lensch. NeRD: Neural reflectance decomposition from image collections. *https://arxiv.org/abs/2012.03918*, 2020. 2, 3, 5, 6, 7, 12, 13

[4] Mark Boss, Varun Jampani, Raphael Braun, Ce Liu, Jonathan T. Barron, and Hendrik P. A. Lensch. Neural-pil: Neural pre-integrated lighting for reflectance decomposition. *CoRR*, abs/2110.14373, 2021. 2, 3

[5] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Proceedings of the Neural Information Processing Systems Conference*, pages 2672–2680, 2014. 3

[6] Michelle Guo, Alireza Fathi, Jiajun Wu, and Thomas Funkhouser. Object-centric neural scene rendering. *arXiv preprint arXiv:2012.08503*, 2020. 2

[7] Wonbong Jang and Lourdes Agapito. Codenerf: Disentangled neural radiance fields for object categories. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 12949–12958, October 2021. 3

[8] Yoonwoo Jeong, Seokjun Ahn, Christopher Choy, Animashree Anandkumar, Minsu Cho, and Jaesik Park. Self-calibrating neural radiance fields, 2021. 2

[9] Kaleido AI GmbH, 2018. https://www.remove.bg/. 6

[10] Petr Kellnhofer, Lars Jebe, Andrew Jones, Ryan Spicer, Kari Pulli, and Gordon Wetzstein. Neural lumigraph rendering. In *CVPR*, 2021. 2

[11] Ira Kemelmacher-Shlizerman and Steven M. Seitz. Face reconstruction in the wild. In *2011 International Conference on Computer Vision*, pages 1746–1753, 2011. 2

[12] Alex Kendall and Yarin Gal. What uncertainties do we need in bayesian deep learning for computer vision? In Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, pages 5574–5584, 2017. 4

[13] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *CoRR*, abs/1412.6980, 2014. 6

[14] Zhengqi Li, Simon Niklaus, Noah Snavely, and Oliver Wang. Neural scene flow fields for space-time view synthesis of dynamic scenes. *https://arxiv.org/abs/2011.13084*, 2020. 1

[15] Shu Liang, Linda G Shapiro, and Ira Kemelmacher-Shlizerman. Head reconstruction from internet photos. In *European Conference on Computer Vision*, pages 360–374. Springer, 2016. 2

[16] Chen-Hsuan Lin, Wei-Chiu Ma, Antonio Torralba, and Simon Lucey. BARF: bundle-adjusting neural radiance fields. *CoRR*, abs/2104.06405, 2021. 2, 13

[17] David Lindell, Julien Martel, and Gordon Wetzstein. AutoInt: Automatic integration for fast neural volume rendering. *https://arxiv.org/abs/2012.01714*, 2020. 2

[18] Lingjie Liu, Jiatao Gu, Kyaw Zaw Lin, Tat-Seng Chua, and Christian Theobalt. Neural sparse voxel fields. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020. 2

[19] Ilya Loshchilov and Frank Hutter. SGDR: stochastic gradient descent with restarts. *CoRR*, abs/1608.03983, 2016. 12

[20] Ricardo Martin-Brualla, Noha Radwan, Mehdi Sajjadi, Jonathan T. Barron, Alexey Dosovitskiy, and Daniel Duckworth. NeRF in the wild: Neural radiance fields for unconstrained photo collections. *https://arxiv.org/abs/2008.02268*, 2020. 2, 3, 4, 6

[21] Moustafa Meshry, Dan B Goldman, Sameh Khamis, Hugues Hoppe, Rohit Pandey, Noah Snavely, and Ricardo Martin-Brualla. Neural rerendering in the wild. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 6878–6887, 2019. 3

[22] Ben Mildenhall, Pratul P Srinivasan, Matthew Tancik, Jonathan T Barron, Ravi Ramamoorthi, and Ren Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. In *European Conference on Computer Vision*, pages 405–421. Springer, 2020. 1, 2, 3, 4, 6, 7, 13, 14

[23] Thomas Neff, Pascal Stadlbauer, Mathias Parger, Andreas Kurz, Chakravarty R. Alla Chaitanya, Anton Kaplanyan, and Markus Steinberger. Donerf: Towards real-time rendering of neural radiance fields using depth oracle networks, 2021. 2

[24] Thu Nguyen-Phuoc, Chuan Li, Lucas Theis, Christian Richardt, and Yong-Liang Yang. Hologan: Unsupervised learning of 3d representations from natural images. *CoRR*, abs/1904.01326, 2019. 3

[25] Michael Niemeyer and Andreas Geiger. GIRAFFE: Representing scenes as compositional generative neural feature fields. *https://arxiv.org/abs/2011.12100*, 2020. 3

[26] Keunhong Park, Utkarsh Sinha, Peter Hedman, Jonathan T. Barron, Sofien Bouaziz, Dan B. Goldman, Ricardo Martin-Brualla, and Steven M. Seitz. Hypernerf: A higher-dimensional representation for topologically varying neural radiance fields. *CoRR*, abs/2106.13228, 2021. 1

[27] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In H. Wallach, H. Larochelle, A. Beygelzimer, F. dÁlché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems 32*, pages 8024–8035. Curran Associates, Inc., 2019. 6, 11

[28] Bui Tuong Phong. Illumination for computer generated pictures. *Communications of the ACM*, 18(6):311–317, 1975. 5, 11

[29] Albert Pumarola, Enric Corona, Gerard Pons-Moll, and Francesc Moreno-Noguer. D-NeRF: Neural radiance fields for dynamic scenes. *https://arxiv.org/abs/2011.13961*, 2020. 1

[30] Ravi Ramamoorthi and Pat Hanrahan. A signal-processing framework for inverse rendering. In Lynn Pocock, editor, *Proceedings of the 28th Annual Conference on Computer Graphics and Interactive Techniques, SIGGRAPH 2001, Los Angeles, California, USA, August 12-17, 2001*, pages 117–128. ACM, 2001. 5, 11

[31] Christian Reiser, Songyou Peng, Yiyi Liao, and Andreas Geiger. Kilonerf: Speeding up neural radiance fields with thousands of tiny mlps. *CoRR*, abs/2103.13744, 2021. 2

[32] Paul-Edouard Sarlin, Daniel DeTone, Tomasz Malisiewicz, and Andrew Rabinovich. Superglue: Learning feature matching with graph neural networks. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2020, Seattle, WA, USA, June 13-19, 2020*, pages 4937–4946. Computer Vision Foundation / IEEE, 2020. 6

[33] Johannes Lutz Schönberger and Jan-Michael Frahm. Structure-from-motion revisited. In *CVPR*, 2016. 2, 4, 6

[34] Katja Schwarz, Yiyi Liao, Michael Niemeyer, and Andreas Geiger. Graf: Generative radiance fields for 3D-aware image synthesis. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020. 3

[35] Noah Snavely, Steven M. Seitz, and Richard Szeliski. Photo tourism: exploring photo collections in 3d. *ACM Trans. Graph.*, 25(3):835–846, 2006. 2

[36] Noah Snavely, Steven M. Seitz, and Richard Szeliski. Modeling the world from internet photo collections. *Int. J. Comput. Vis.*, 80(2):189–210, 2008. 2

[37] Pratul Srinivasan, Boyang Deng, Xiuming Zhang, Matthew Tancik, Ben Mildenhall, and Jonathan T. Barron. NeRV: Neural reflectance and visibility fields for relighting and view synthesis. *https://arxiv.org/abs/2012.03927*, 2020. 3, 8

[38] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł ukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. 12

[39] Qianqian Wang, Zhicheng Wang, Kyle Genova, Pratul Srinivasan, Howard Zhou, Jonathan T. Barron, Ricardo Martin-Brualla, Noah Snavely, and Thomas Funkhouser. Ibrnet: Learning multi-view image-based rendering, 2021. 2

[40] Zirui Wang, Shangzhe Wu, Weidi Xie, Min Chen, and Victor Adrian Prisacariu. NeRF–: Neural radiance fields without known camera parameters. *https://arxiv.org/abs/2102.07064*, 2021. 2, 4

[41] Wenqi Xian, Jia-Bin Huang, Johannes Kopf, and Changil Kim. Space-time neural irradiance fields for free-viewpoint video. *https://arxiv.org/abs/2011.12950*, 2020. 1

[42] Christopher Xie, Keunhong Park, Ricardo Martin-Brualla, and Matthew Brown. Fig-nerf: Figure-ground neural radiance fields for 3d object category modelling. In *International Conference on 3D Vision (3DV)*, 2021. 3

[43] Bangbang Yang, Yinda Zhang, Yinghao Xu, Yijin Li, Han Zhou, Hujun Bao, Guofeng Zhang, and Zhaopeng Cui. Learning object-compositional neural radiance field for editable scene rendering. In *International Conference on Computer Vision (ICCV)*, October 2021. 2

[44] Lin Yen-Chen, Pete Florence, Jonathan T. Barron, Alberto Rodriguez, Phillip Isola, and Tsung-Yi Lin. iNeRF: Inverting neural radiance fields for pose estimation. *https://arxiv.org/abs/2012.05877*, 2020. 2

[45] Alex Yu, Ruilong Li, Matthew Tancik, Hao Li, Ren Ng, and Angjoo Kanazawa. Plenoctrees for real-time rendering of neural radiance fields. *CoRR*, abs/2103.14024, 2021. 2

[46] Jason Y. Zhang, Gengshan Yang, Shubham Tulsiani, and Deva Ramanan. Ners: Neural reflectance surfaces for sparse-view 3d reconstruction in the wild. *CoRR*, abs/2110.07604, 2021. 3

[47] Kai Zhang, Fujun Luan, Qianqian Wang, Kavita Bala, and Noah Snavely. Physg: Inverse rendering with spherical gaussians for physics-based material editing and relighting. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2021, virtual, June 19-25, 2021*, pages 5453–5462. Computer Vision Foundation / IEEE, 2021. 3

[48] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric, 2018. 13

[49] Xiuming Zhang, Pratul P Srinivasan, Boyang Deng, Paul Debevec, William T Freeman, and Jonathan T Barron. NeRFactor: Neural Factorization of Shape and Reflectance Under an Unknown Illumination. *arXiv preprint arXiv:2106.01970*, 2021. 2, 3, 5, 6, 8

A. Implementation Details

A.1. SH Rendering Model

Spherical Harmonics (SH) represents a group of basis functions defined on the sphere surface, commonly used for factorizing functions and fast integration for multiplying functions. A Spherical Harmonic $Y_{lm}(\theta, \phi)$ of index l, m is defined as:

$$Y_{lm}(\theta,\phi) = \sqrt{\frac{2l+1}{4\pi}\frac{(l-m)!}{(l+m)!}} P_l^m(\cos\theta) e^{Im\phi}, \quad (8)$$

where $0 \leq l \leq +\infty, -l \leq m \leq l$, and $P_l^m(\cos\theta)e^{Im\phi}$ are the associated Legendre polynomials.

Below we describe our rendering pipeline using Spherical Harmonics. Our model aims to calculate the single-bounce light reflections on the object surface from a spherical environment map L, where the light transport equation is defined as:

$$B(\boldsymbol{n}, \boldsymbol{\omega}_o) = \int_{\boldsymbol{\omega}_i \in \Omega^+} L(\boldsymbol{\omega}_i)\rho(\boldsymbol{\omega}_o, \boldsymbol{\omega}_i)(\boldsymbol{n} \cdot \boldsymbol{\omega}_i) d\boldsymbol{\omega}_i, \quad (9)$$

where $\boldsymbol{n}, \boldsymbol{\omega}_i, \boldsymbol{\omega}_o$ are directions of surface normal, incoming light, and outgoing light, Ω^+ is the upper hemisphere above the surface, and $B(\boldsymbol{n}, \boldsymbol{\omega}_o), L(\boldsymbol{\omega}_i), \rho(\boldsymbol{\omega}_i, \boldsymbol{\omega}_o)$ are the outgoing light towards direction $\boldsymbol{\omega}_o$, the incoming light from direction $\boldsymbol{\omega}_i$, and the bidirectional reflectance distribution function (BRDF) between $\boldsymbol{\omega}_i$ and $\boldsymbol{\omega}_o$, respectively.

According to [30], functions L and ρ can be approximated by a group of SHs $Y_{lm}(\boldsymbol{\omega})$ as:

$$L(\boldsymbol{\omega}_i) \approx \sum_{l,m} L_{lm} Y_{lm}(\boldsymbol{\omega}_i), \quad (10)$$

$$\rho(\boldsymbol{\omega}_i, \boldsymbol{\omega}_o) \approx \sum_{l,m}\sum_{p,q} \rho_{lm,pq} Y^*_{lm}(\boldsymbol{\omega}_i) Y_{pq}(\boldsymbol{\omega}_o), \quad (11)$$

where $0 \leq \{l, p\} \leq +\infty, -l \leq m \leq l, -p \leq q \leq p$, Y^*_{lm} is the conjugate of Y_{lm}, and L_{lm}, $\rho_{lm,pq}$ are coefficients calculated by applying an integration on the multiplication of functions L, ρ and the SHs.

If the BRDF ρ is isotropic, we can reduce its number of coefficient indices to three, denoted as ρ_{lpq}. The outgoing light field B can thus be approximated as:

$$B(\boldsymbol{n}, \boldsymbol{\omega}_o) \approx \sum_{l,m,p,q} B_{lmpq} C_{lmpq}(\boldsymbol{n}, \boldsymbol{\omega}_o), \quad (12)$$

where $B_{lmpq} = \Lambda_l L_{lm} \rho_{lpq}$, $\Lambda_l = \sqrt{4\pi/(2l+1)}$ is a normalizing constant, and $C_{lmpq}(\boldsymbol{n}, \boldsymbol{\omega}_o)$ is a set of basis functions which are not discussed in detail here.

If the BRDF is independent of $\boldsymbol{\omega}_o$, we can further simplify Eq. 12 by removing $\boldsymbol{\omega}_o$ as:

$$B(\boldsymbol{n}) \approx \sum_{l,m} B_{lm} Y_{lm}(\boldsymbol{n}), \quad (13)$$

where $B_{lm} = \Lambda_l L_{lm} \rho_{l00} \doteq \Lambda_l L_{lm} \rho_l$.

We use the Phong BRDF model [28] to represent the object material in all our experiments, which is defined as:

$$\rho(\boldsymbol{\omega}_i, \boldsymbol{\omega}_o) = \frac{K_d}{\pi}(\boldsymbol{\omega}_i \cdot \boldsymbol{n}) + \frac{K_s(g+1)}{2\pi}(\boldsymbol{\omega}_i \cdot \boldsymbol{\omega}_r)^g, \quad (14)$$

where K_d, K_s, g are parameters of the base color, specularity, and glossiness, and $\boldsymbol{\omega}_r$ is the reflection of $\boldsymbol{\omega}_o$. We calculate the two terms in Eq. 14 separately.

The first term is also known as the Lambertian BRDF. It has been demonstrated that calculating Eq. 13 with $l <= 2$ can capture more than 99% of the reflected radiance of this term. Let $A_l = \Lambda_l \rho_l$ be the normalized coefficient of term $(\boldsymbol{\omega}_i \cdot \boldsymbol{n})$, we have $A_0 = 3.14, A_1 = 2.09, A_2 = 0.79$. Bringing them into Eq. 13, we can calculate the Lambertian term by querying the value of each SH at $\boldsymbol{n}$, calculating the weighted sum, and finally multiplying it with K_d/π.

As for the second term, we adopt the method from [30], in which we replace $\boldsymbol{n}$ with $\boldsymbol{\omega}_r$ in Eq. 12, thus making it independent of $\boldsymbol{\omega}_o$ and reducible to Eq. 13. In this case, the approximation of the BRDF coefficients is given as:

$$\Lambda_l \rho_l \approx \exp(-\frac{l^2}{2g}). \quad (15)$$

The remaining steps are then the same as the first term.

Our renderer is implemented in PyTorch [27] and is fully differentiable. In all our experiments, we set $l \leq 3$, which leads to 16 light coefficients L_{lm} for each color channel to optimize (in total $16 \times 3 = 48$ parameters). Parameters K_d, K_s are limited to $[0, 1]$, and $g \in [1, +\infty]$. To reduce the ambiguity, we assume white specular highlights, and thus setting the channels of K_s to 1.

Tone-Mapping. Since our renderer calculates the radiance in linear HDR space, we also apply a tone-mapping process to the rendered results. It is simply defined as:

$$\mathcal{T}_k(x) = x^{(1/\gamma_k)}, \quad (16)$$

where γ_k is a trainable parameter assigned to image $\mathcal{I}_k$, and is initialized from 2.4, which is the default value of common sRGB curves. On the other hand, we do not apply exposure compensation nor white balance to our renderer's output, assuming that our SH renderer can automatically fit these variances during the optimization.

A.2. Losses

Here we explain the losses in more details. Firstly, the color reconstruction loss $\mathcal{L}_c$ and the transient regularity loss

$\mathcal{L}_{\text{tr}}$ introduced in Sec. 3.3 in the paper are defined as:

$$\mathcal{L}_{\text{c}}(\boldsymbol{r}) = \frac{\|C_k(\boldsymbol{r}) - \mathcal{I}_k(\boldsymbol{r})\|_2^2}{2\beta_k(\boldsymbol{r})^2} + \frac{\log(\beta_k(\boldsymbol{r})^2)}{2}, \tag{17}$$

$$\mathcal{L}_{\text{tr}}(\boldsymbol{r}) = \frac{1}{N_p}\sum_{i=1}^{N_p} \sigma_k^{(\tau)}(\boldsymbol{x}_i), \tag{18}$$

where $\boldsymbol{r}$ is a ray from image $\mathcal{I}_k$ and $\boldsymbol{x}_i$ are the sample points along $\boldsymbol{r}$. $\beta_k(\boldsymbol{r})$ is the uncertainty along the ray $\boldsymbol{r}$, which integrates the uncertainty predictions at all sample points.

During the training of the rendering model, we also employ a regularity loss L_{reg} to prevent improbable solutions. This loss is defined as:

$$\begin{aligned}\mathcal{L}_{\text{reg}} = \lambda_{\text{spec}}\|K_s\|_2^2 + \lambda_{\text{gamma}}\frac{1}{N}\sum_{k=1}^{N}\|\gamma_k - 2.4\|_2^2 \\ + \lambda_{\text{light}}\frac{1}{N_t}\sum_{t=1}^{N_t}\|\text{ReLU}(-L_{k_t}(\boldsymbol{\omega}_t) - \tau_{\text{light}})\|_2^2,\end{aligned} \tag{19}$$

where coefficients $\lambda_{\text{spec}}, \lambda_{\text{gamma}}, \lambda_{\text{light}}$ are set to 0.1, 5, and 5, respectively. The last term is for light regularization, designed to prevent negative values (lower than $-\tau_{\text{light}}$, with τ_{light} set to 0.01) in the SH lighting model, which may happen during training due to over-fitted shadows. For each iteration, we randomly sample N_t incoming light directions $\boldsymbol{\omega}_t$ and image indices k_t, and evaluate the corresponding incoming light values for the loss calculation. N_t is set to be identical to the batch size in our experiments.

A.3. Network Structure

In our first stage, the geometry network (Sec. 3.3), the input position vector $\boldsymbol{x}$ is embedded using the positional encoding method introduced in [38], then fed into an 8-layer MLP with the hidden vector dimension of 256. The resulting embedding $\boldsymbol{z}_{\boldsymbol{x}}$, is then fed into three branches: a branch consisting of one layer to predict static density $\sigma^{(s)}$; a branch consisting of one layer to predict static color $c_k^{(s)}$, which also takes the positional-embedded view direction $\boldsymbol{d}$ and appearance embedding $\boldsymbol{z}_k^{(a)}$ as input; and a branch of another 4-layer MLP with a hidden vector dimension of 128, followed by several output layers to predict transient density $\sigma_k^{(\tau)}$, transient color $\boldsymbol{c}_k^{(\tau)}$ and uncertainty β_k, where the transient embedding $\boldsymbol{z}_k^{(\tau)}$ is also provided as input.

Our second stage, the rendering network (Sec. 3.5), shares the same structure as the first stage on most components, except the branch of static color prediction. This branch is replaced by a new 4-layer MLP with the hidden vector dimension of 128, which takes $\boldsymbol{x}$ and $\boldsymbol{z}_{\boldsymbol{x}}$ as input, followed by several output layers to generate normal $\boldsymbol{n}$, base color K_d, specularity K_s, and glossiness g.

We choose ReLU as the activation function for all intermediate layers. For the outputs layers, we adopt SoftPlus for density functions, uncertainty, and glossiness; Sigmoid for static/transient/base color and specularity; and a vector normalizing layer for normal estimation.

In addition to our network parameters, we also jointly optimize the light coefficients $L_{k,lm}$, the camera parameters $(\delta_R, \delta_t, \delta_f)_k$, and the tone-mapping parameter γ_k for each image $\mathcal{I}_k$.

Table 3. **Details of our datasets.** We split our datasets into three categories based on their sources (from [3], self-captured, and collected from the Internet). In addition to the datasets seen in the paper, we collected another one, *Bust*, from the Internet, which is shown in the sample images below and the supplementary video.

Dataset	Image #	Train #	Test #	λ in DEL
From [3]				
Cape	119	111	8	1
Head	66	62	4	1
Gnome	103	96	7	0.1
MotherChild	104	97	7	1
Self-Captured				
Figure	49	43	6	0.1
Milk	43	37	6	1
TV	40	35	5	1
From the Internet				
Gnome2	35	32	3	1
Dog	36	33	3	1
Bust	41	38	3	1

A.4. Dataset & Training Details

Tab. 3 lists the numbers of images and configurations of our datasets. Since the controllable parameter λ in our depth extraction layer (DEL) is not fixed for all scenes, we also list its values in the rightmost column of the table. Besides the datasets in the table, we also trained our model on the synthetic datasets from [3] (*Globe*, *Chair*) for material validation. More details are explained in Sec. E

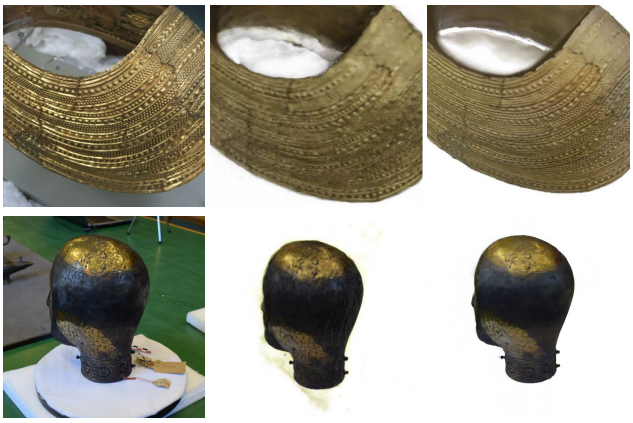
We generate and store rays for all pixels from the input image before training starts. At the beginning of each epoch, we use the foreground masks to ensure that the number of the chosen background rays does not exceed the foreground rays by more than a factor of 2, and then concatenate and shuffle the background and foreground rays together.

In the first stage, we decay the learning rate by a factor of 0.3 at intervals of 10 epochs. In the second stage, we use the cosine annealing schedule [19] with $T_{\text{max}} = 10$ to reduce the learning rate, as the training epoch is relatively small.

Since the SfM pipeline of COLMAP also produces a sparse point cloud of the target object while solving camera poses, we further use them to help train our model. We generate a coarse bounding box of the object based on the points, and only sample ray points inside the bounding box.

Table 4. **Comparison with NeRD on real datasets.** We present results in both static illumination (Cape) and varying illumination (Head, MotherChild, Gnome). As the code of NeRD [3] is not published at the time of submission, we directly copy the results from their paper and mark them using an asterisk. We also report our results on NeRF here as reference. We highlight the best and second best results in each column in orange and yellow.

Methods	Static Illum.		Varying Illum.	
	PSNR↑	SSIM↑	PSNR↑	SSIM↑
NeRF [22]*	23.34	0.85	20.11	0.87
NeRF-A*	22.87	0.83	26.36	0.94
NeRD [3]*	23.86	0.88	25.81	0.95
NeRF	22.95	0.78	24.31	0.90
Ours-Geom	24.70	0.83	27.96	0.93
Ours-Full	23.47	0.80	27.16	0.92



Ground Truth NeRD [3] Ours

Figure 10. **Qualitative comparison with NeRD**. Results from NeRD are copied from the paper.

In contrast, the data from [3] are captured in the same scene, and the background is also used in their camera registration, making this optimization infeasible in their approach.

B. Comparisons with NeRD

In this section, we compare our model with another material decomposition approach, NeRD [3], quantitatively and qualitatively. As mentioned in our main paper, since the code, models, and key evaluation details of NeRD are not publicly available at the time of submission, it is impossible to perform a perfectly fair comparison with NeRD under identical settings. However, we can still conduct reasonable evaluations and comparisons, given the fact that their datasets are available.

In the quantitative evaluations, we use the same training/testing splits of the datasets as in NeRD, and apply the same testing strategy that optimize the lighting parameters with fixed network parameters, in order to fit the unknown lighting in testing images. The results are shown in Tab. 4. Our model achieves better results with PSNR and competitive results with SSIM. We also train and test NeRF [22] on the same dataset with the same setting as a reference.

We also show our qualitative results in Fig. 10. Thanks to the novel designs of our model, our model achieves better results with smoother geometry, cleaner object boundaries, and sharper textures compared to results shown in NeRD.

Aside from the aforementioned evaluations, we would like to further discuss the intrinsic differences between our work and NeRD: First of all, our work aims to solve reconstruction and rendering on objects from unrestricted and multi-source online image collections. This leads to many challenges such as inaccurate camera poses, ill-conditioned illuminations and transient occlusions, and our model is designed mainly based on how to tackle these problems. On the other hand, NeRD only claimed to handle "image collections", and all of their experiments are conducted on data captured from single source, using the same set of cameras under similar environmental conditions. We strongly believe our model can ourperform NeRD on datasets collected from the Internet.

From a technical standpoint, our model contains many novel features that are not present in NeRD, *i.e.* the transient model, the expected-depth-based sampling, and the Depth Extraction Layer. Among all of these differences, we want to highlight one major difference that our renderer is based on Spherical Harmonics while NeRD uses Spherical Gaussians. We employ Spherical Harmonics following the approach of [16], which jointly optimizes camera poses during NeRF training. It proposes the idea that restricting the parameters and functions in a joint system to low-frequency components can help prevent the joint optimization from falling into local minima. As the structure of Spherical Harmonics makes it easy to control their frequency, we believe that using lower-order Spherical Harmonics can make our system more robust with unrestricted inputs.

C. Comparisons with NeRF

In Sec. 4.2 of the main paper, we provide the mean results of comparisons with NeRF [22] on our evaluation datasets. In Tab. 5, we provide a detailed breakdown of the results per object. We also include the LPIPS (Learned Perceptual Image Patch Similarity) [48] metric, as it correlates well with perceptual quality of images to human observers.

As shown, our model achieves better performance on almost all objects even without optimization, except one dataset Head. We think this is because Head is a relatively simple case where the object is rotating in front of the camera of a fixed pose. Since NeRF has fewer components and parameters to optimize, it is more likely to converge to a sharper geometry and get better results on testing in such a trivial scenario.

Table 5. **Comparison with NeRF**. We highlight the best and second best results in each column, using orange and yellow, respectively. The last column contains the mean result across all target objects.

Methods	Cape			Gnome			Head			MotherChild		
	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓
NeRF [22]	22.95	0.78	0.218	18.73	0.82	0.240	29.13	0.92	0.140	25.08	0.95	0.106
Ours-Geom w/o Opt	23.22	0.82	0.180	26.95	0.89	0.120	26.37	0.92	0.136	23.03	0.95	0.068
Ours-Full w/o Opt	22.82	0.79	0.198	25.30	0.87	0.132	26.08	0.90	0.146	25.69	0.96	0.069
Ours-Geom	24.70	0.83	0.178	28.11	0.89	0.119	26.80	0.92	0.136	28.98	0.97	0.058
Ours-Full	23.47	0.80	0.197	26.11	0.88	0.129	26.35	0.91	0.145	29.02	0.97	0.062
Methods	Milk			Figure			TV			**Mean**		
	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓
NeRF [22]	19.40	0.92	0.145	21.54	0.90	0.159	19.03	0.90	0.145	22.266	0.884	0.164
Ours-Geom w/o Opt	21.41	0.94	0.059	22.89	0.92	0.121	20.31	0.92	0.114	23.454	0.909	0.114
Ours-Full w/o Opt	23.00	0.95	0.066	23.69	0.92	0.133	21.88	0.92	0.122	24.065	0.901	0.124
Ours-Geom	27.51	0.96	0.052	24.41	0.93	0.118	25.06	0.93	0.110	26.510	0.919	0.110
Ours-Full	28.87	0.95	0.056	24.72	0.92	0.130	26.52	0.93	0.107	26.437	0.909	0.118

Table 6. **Ablation study with fewer images**. We highlight the best and second best results in each column in orange and yellow.

Methods	Full			w/ 20 Images			w/ 10 Images		
	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓
NeRF [22]	19.03	0.90	0.145	16.52	0.89	0.167	17.51	0.88	0.244
Ours-Geom	25.06	0.93	0.110	24.76	0.92	0.119	23.54	0.91	0.148
Ours-Full	26.52	0.93	0.107	26.26	0.92	0.119	25.26	0.92	0.147

D. Additional Ablation Studies

We show two additional ablation studies on our model in this section to support the ablative research in our paper.

The first experiment trys to help understand the effect of reducing training image number on our model. In this experiment, we compare our full model with two variants, whose training set are reduced to 20 and 10 images respectively, and the results are shown in Tab. 6. We also tested the same settings on NeRF as baseline. As scores of both model are both decreased when training images get fewer, our model are less effected than NeRF, especially on LPIPS score. This result proves that our model is able to keep the output accurate in a perceptual manner even if only sparse training images are given.

The second experiment is qualitative and focuses on the sampling strategy with expected depth. As shown in Fig. 11, the hybrid method using depth variance which explained in the paper helps eliminate the aliasing effect along the object's border, while the loss of training efficiency is acceptable. We also show the depth variance map filtered by the threshold τ_d, where rays from white pixels are sampled with all points and others only uses the expected depth. In all of our experiments, τ_d is set to $(d_f - d_n)/5000$, where d_n, d_f are the depths of the near/far planes of the camera.

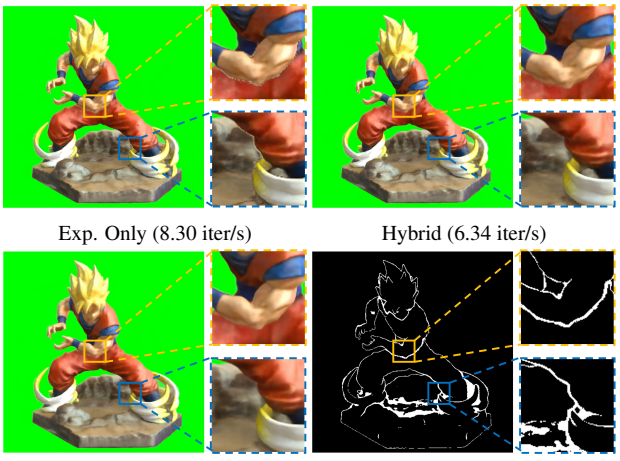


Figure 11. **Ablation study on our sampling strategy.** From top to bottom, left to right: result with samples on expected depth only; result using hybrid sampling based on depth variance; result with all sample points; the depth variance map filtered by τ_d. We also show the training speed of each model in iterations per second.

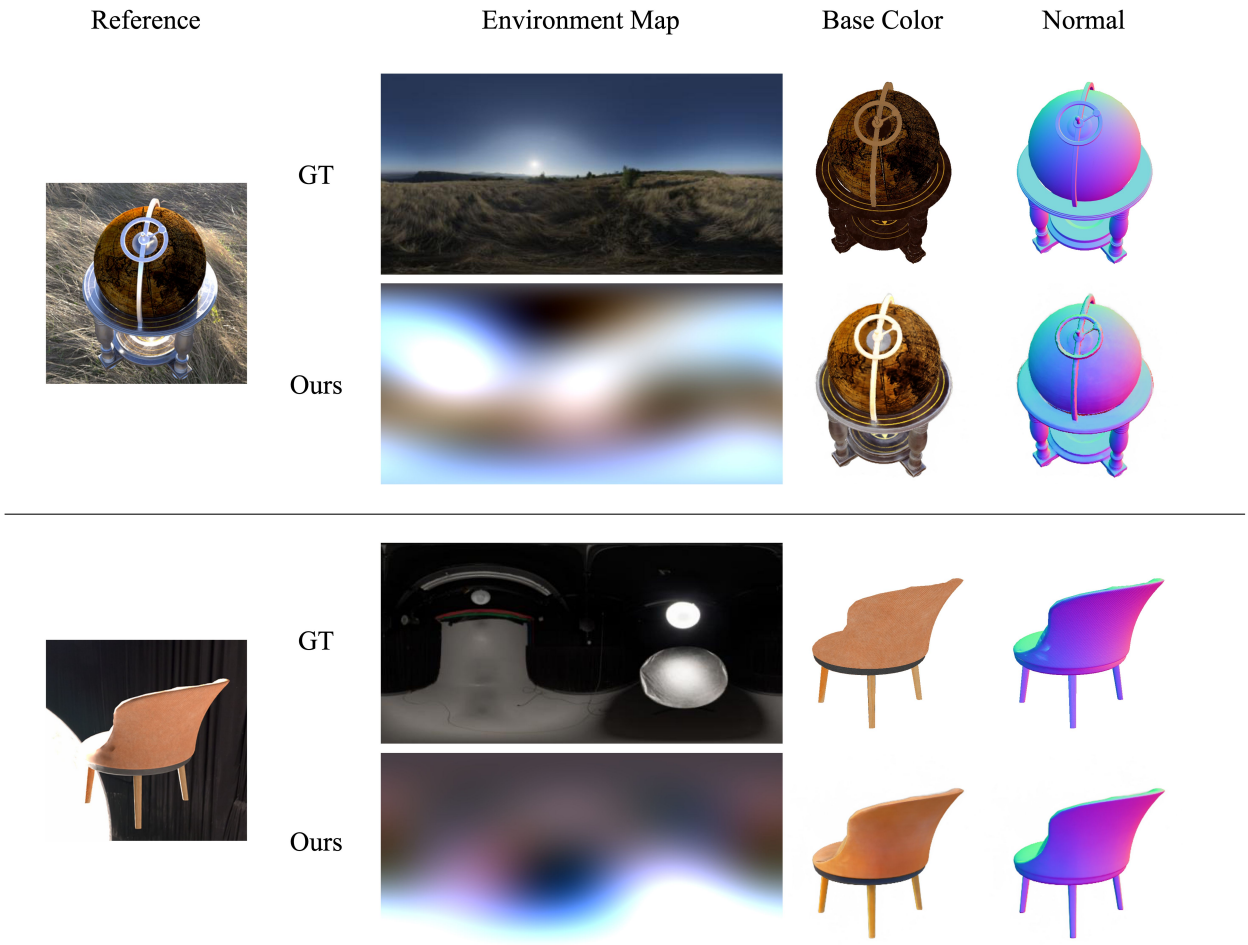


Figure 12. **Results on synthetic datasets**. As our BRDF model is different from the ground truth, we only show components that exist in both models, including the Base Color, Normal, and Environment Map.

E. More Results

While all the aforementioned experiments are conducted on real datasets, we also trained our model on synthetic data to validate the quality of the material prediction. In Fig. 12, we show our output materials, normal and environment maps along with the ground truth.

Even through our BRDF model and lighting parameters differ from the ground truth used when rendering these synthetic images, our method is still able to reconstruct these properties reasonably well. This further demonstrates that our approach is robust to a wide variety of input capture or image acquisition conditions.

Lastly, we provide results from our trained model in different use cases. Fig. 13 shows synthesized novel views of the target objects generated by our model; Fig. 14 shows our objects' predicted materials and normals; and Fig. 15 shows rendered results of objects relit in new environments.

We note that, for the bust of Nefertiti seen in Figs. 13 and 15 (last columns), capturing our own input images for object capture would be infeasible, given the manner in which this object is stored in a remote and secure location.

We also provide animated results for these objects in our supplementary video.

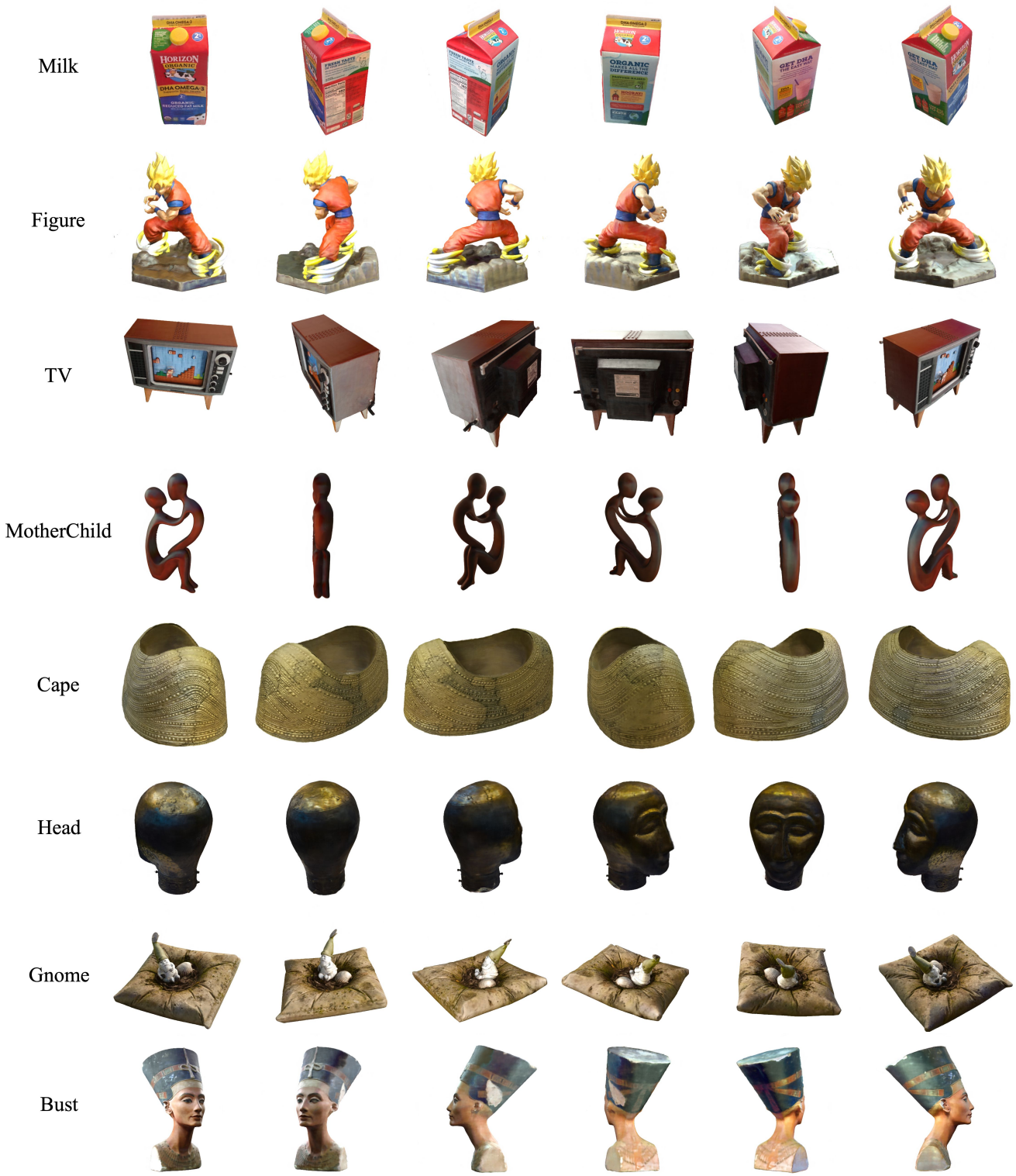


Figure 13. **Synthesized novel views generated using our approach**.

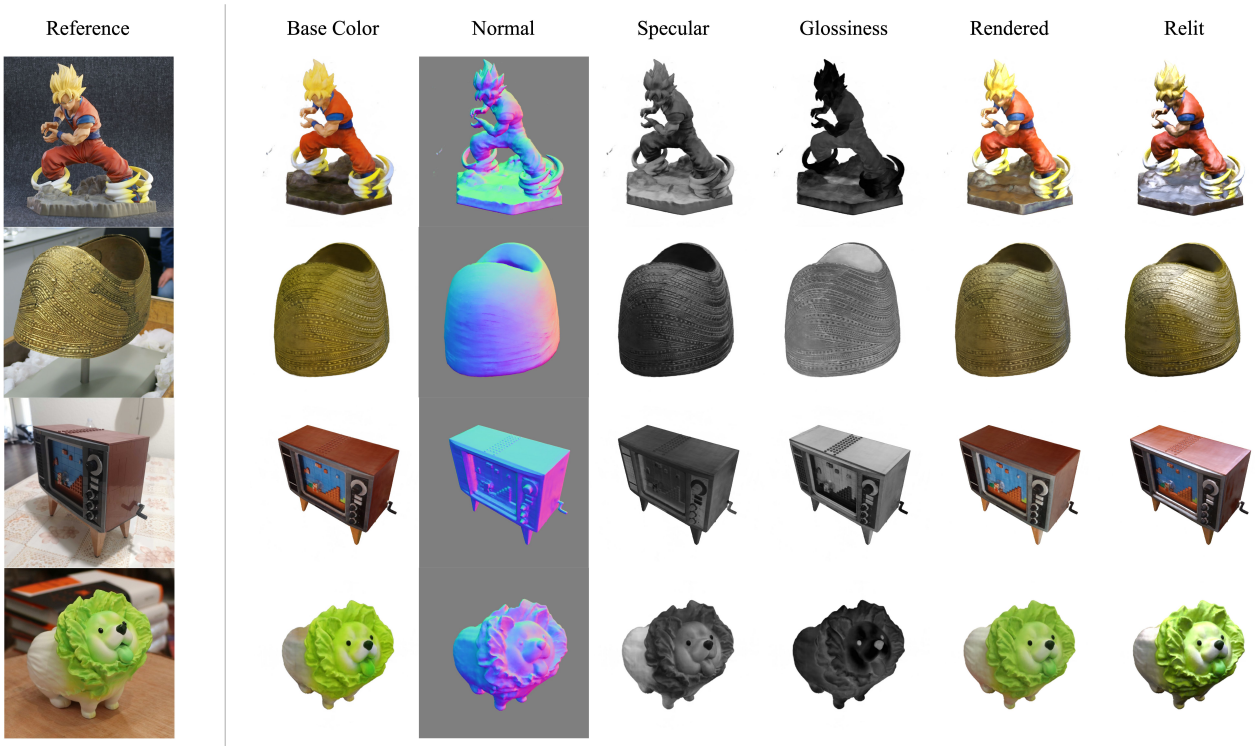


Figure 14. **A showcase of additional material decomposition results using our approach**.

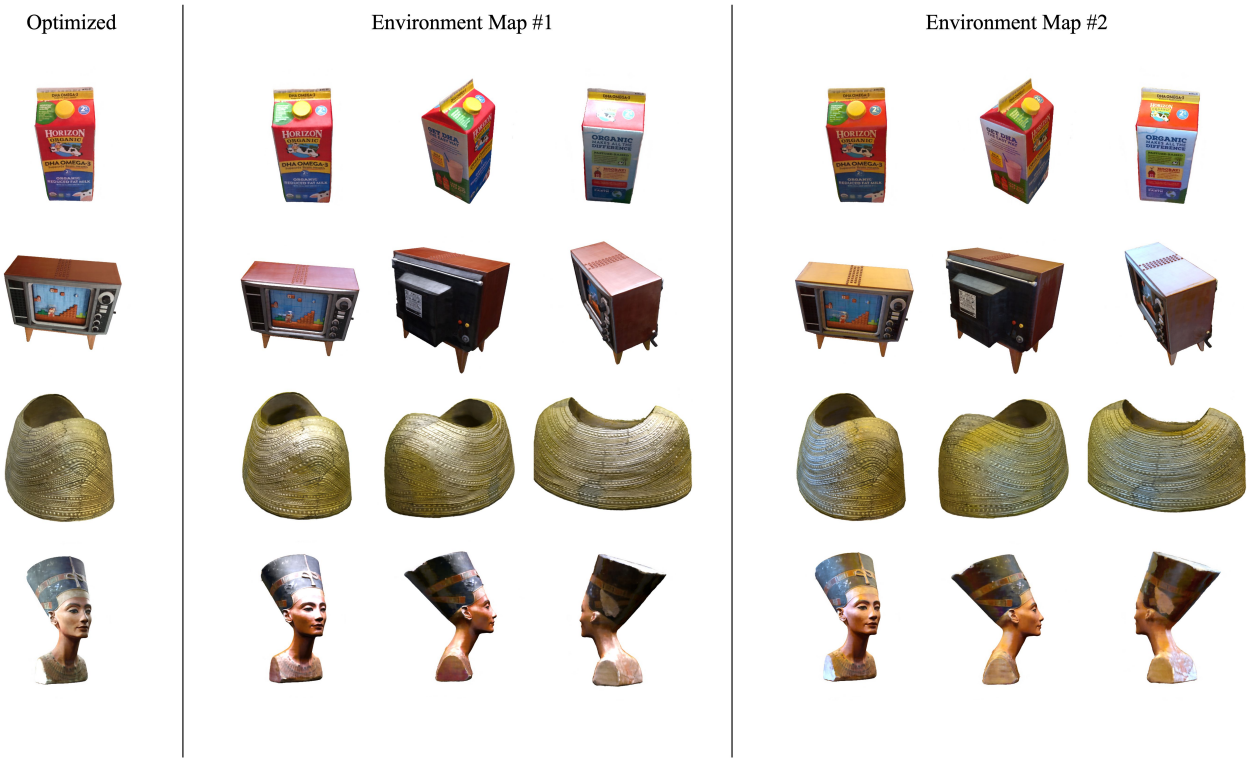


Figure 15. **Target objects rendered in new environments using our approach**.